

Capítulo 7. Búsqueda y resolución algorítmica de problemas

Buscar es convertir una pregunta del tipo «¿qué secuencia de decisiones conduce desde esta situación hasta una situación aceptable?» en un procedimiento sistemático. Un planificador de recorridos, un rompecabezas, una asignación de turnos y una secuencia de inspecciones parecen problemas distintos; sin embargo, todos pueden describirse mediante estados, acciones, objetivos y costos. La representación establece qué soluciones existen. La estrategia de búsqueda establece cuáles se examinan primero y qué recursos se consumen para encontrarlas.

Este capítulo estudia problemas deliberativos, estáticos y conocidos durante la búsqueda. En ese marco, una acción tiene un resultado determinado y el algoritmo puede evaluar si alcanzó el objetivo. La incertidumbre, el cambio del entorno y las decisiones repetidas se abordarán en capítulos posteriores. Esta separación es importante: una ruta óptima en un modelo fijo no es necesariamente la mejor decisión cuando los tiempos cambian o se desconocen.

Al finalizar el capítulo, se espera que el lector pueda:

- formular un problema mediante un espacio de estados y distinguir estado, nodo y camino;
- implementar conceptualmente una búsqueda con frontera y conjunto explorado;
- seleccionar entre búsqueda en anchura, profundidad, profundidad limitada, profundización iterativa y costo uniforme;
- razonar sobre completitud, optimalidad, tiempo y memoria bajo supuestos explícitos;
- diseñar y evaluar heurísticas para búsqueda voraz y A^* ;
- reconocer cuándo conviene una búsqueda local o una formulación como problema de satisfacción de restricciones;
- interpretar correctamente un grafo de movilidad construido con pares origen-destino observados.

La exposición emplea pseudocódigo independiente del lenguaje. Una operación como INSERTAR no prescribe una biblioteca ni una estructura concreta: expresa el comportamiento que debe conservar cualquier implementación.

7.1. Espacios de estados y procesos de búsqueda

Un **problema de búsqueda** puede representarse formalmente como

$$P = (S, A, T, s_0, G, c),$$

donde S es el conjunto de estados; $A(s)$, las acciones aplicables en s ; $T(s, a)$, el estado resultante; s_0 , el estado inicial; $G(s)$, un predicado que reconoce objetivos; y $c(s, a, s')$, el costo de pasar de s a s' . Una solución es una secuencia de acciones aplicables que lleva desde s_0 hasta algún estado que satisface G . Su costo es la suma de los costos de sus transiciones.

Esta definición no obliga a enumerar S . En problemas reales, el espacio suele describirse de manera implícita mediante una función sucesora. La búsqueda

genera solo la parte necesaria. Antes de elegir un algoritmo, por tanto, hay que decidir qué información integra un estado, qué acciones son legales y cuándo dos descripciones representan la misma situación.

7.1.1. Árboles y grafos

Un **grafo** es un par $G = (V, E)$, con un conjunto de vértices V y un conjunto de aristas E . En un grafo dirigido, cada arista es un par ordenado (u, v) : poder avanzar de u a v no implica poder volver de v a u . En un grafo no dirigido, una arista expresa una relación simétrica. Si a cada arista se le asocia un valor $w(u, v)$, se obtiene un grafo ponderado. El peso puede representar distancia, tiempo, dinero, riesgo o una combinación cuyo significado debe declararse.

Un **árbol** es un grafo conectado y sin ciclos. Si se elige una raíz, cada nodo salvo la raíz tiene un único padre y existe un único camino simple desde la raíz hasta cualquier nodo. El árbol es útil para visualizar el proceso de búsqueda, pero el dominio subyacente puede ser un grafo. Esta distinción evita una confusión frecuente:

- el **grafo de estados** contiene una vez cada estado y todas las transiciones legales;
- el **árbol de búsqueda** contiene las distintas maneras en que el algoritmo llega a estados, por lo que un mismo estado puede aparecer en varios nodos.

Considérese un espacio con transiciones $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$ y $D \rightarrow A$. El grafo tiene cuatro estados. El árbol de búsqueda puede incluir un nodo para D alcanzado por A, B, D , otro para D alcanzado por A, C, D y, si no se controlan ciclos, infinitas ramas como $A, B, D, A, B, D, \dots$. El dominio es finito, pero el árbol generado no tiene por qué serlo.

No todo grafo almacenado corresponde a un espacio de estados completo. Una tabla de conexiones observadas puede omitir transiciones posibles; un mapa agregado puede unir zonas sin describir el trayecto interior. Las propiedades del algoritmo se refieren al modelo proporcionado, no a una realidad que el modelo no representa.

7.1.2. Nodos, estados y caminos

Un **estado** describe una configuración relevante del problema. Un **nodo de búsqueda** es un registro creado por el algoritmo. Habitualmente contiene:

$$n = (\text{estado}, \text{padre}, \text{acción}, g, \text{profundidad}),$$

donde $g(n)$ es el costo acumulado desde la raíz. Dos nodos pueden contener el mismo estado y diferir en su padre, profundidad o costo. Identificar nodos por estado sin considerar el costo puede ser incorrecto cuando existen caminos alternativos.

Un **camino** es una secuencia $\langle s_0, s_1, \dots, s_k \rangle$ tal que cada par consecutivo está conectado por una transición legal. Su longitud es k , es decir, el número de aristas. Su costo es

$$g = \sum_{i=0}^{k-1} c(s_i, a_i, s_{i+1}).$$

Longitud y costo solo coinciden cuando todas las acciones cuestan una unidad. Un camino de dos aristas puede costar más que uno de cinco. Esta diferencia explica por qué la búsqueda en anchura minimiza cantidad de pasos, mientras que la búsqueda de costo uniforme minimiza costo acumulado.

La secuencia solución se recupera siguiendo punteros al padre desde el nodo objetivo hasta la raíz y luego invirtiendo el orden. Un algoritmo que devuelve únicamente el estado objetivo no ha reconstruido todavía el plan. También conviene separar tres magnitudes: **generar** un nodo es producirlo como sucesor; **insertarlo** es admitirlo en la frontera; **expandirlo** es retirarlo de la frontera y generar sus sucesores. Los informes experimentales deben indicar cuál se cuenta.

7.1.3. Frontera y conjunto explorado

La **frontera** reúne nodos generados pendientes de expansión. La política para extraer un nodo define la estrategia: una cola FIFO produce anchura; una pila LIFO, profundidad; una cola de prioridad por g , costo uniforme; y una cola por $g + h$, A*. El **conjunto explorado** registra estados ya expandidos para evitar trabajo redundante y ciclos.

El siguiente esquema abstrae una búsqueda en grafo. CLAVE devuelve la representación canónica con la que se reconoce un estado repetido.

```
BUSQUEDA-EN-GRAFO(problema, frontera):
  raiz <- NODO(problema.estado_inicial)
  INSERTAR(frontera, raiz)
  mejor_costo[CLAVE(raiz.estado)] <- 0
  explorado <- conjunto vacio

  mientras frontera no este vacia:
    nodo <- EXTRAER(frontera)
    si la entrada de nodo es obsoleta respecto de mejor_costo:
      continuar
    si ES-OBJETIVO(nodo.estado):
      devolver RECONSTRUIR-CAMINO(nodo)
      agregar CLAVE(nodo.estado) a explorado

    para cada (accion, estado_hijo, costo_paso) en SUCESORES(nodo.estado):
      nuevo_g <- nodo.g + costo_paso
      clave <- CLAVE(estado_hijo)
      si clave no fue descubierta o nuevo_g < mejor_costo[clave]:
        mejor_costo[clave] <- nuevo_g
        hijo <- NODO(estado_hijo, nodo, accion, nuevo_g)
        INSERTAR-O-ACTUALIZAR(frontera, hijo)

  devolver fracaso
```

El esquema es deliberadamente general. Para anchura con costos unitarios basta recordar si un estado ya fue descubierta; para costo uniforme y A* hay que

conservar el mejor g conocido. Marcar un estado como descubierto al generarlo evita duplicados en la frontera, pero no autoriza a descartar un camino posterior más barato en algoritmos sensibles al costo.

Una frontera bien definida también resuelve empates. Dos nodos pueden tener la misma prioridad; usar orden de inserción, menor heurística o una regla lexicográfica cambia el número y orden de expansiones, aunque no siempre las garantías. La regla debe ser estable y documentada para que un experimento sea reproducible.

7.1.4. Búsqueda en árbol y búsqueda en grafo

La **búsqueda en árbol** no recuerda estados previamente alcanzados. Cada sucesor crea un nodo nuevo, aunque su estado haya aparecido antes. Requiere menos mecanismos de control y puede ser adecuada cuando el espacio es realmente un árbol o cuando cada secuencia conduce a un estado único. En grafos con rutas convergentes desperdicia trabajo; con ciclos puede no terminar.

La **búsqueda en grafo** incorpora detección de repetidos. Su ventaja es reducir expansiones; su costo es almacenar claves de estados y decidir qué hacer cuando aparece una ruta mejor. Esa decisión depende de la estrategia:

- con anchura y costos unitarios, la primera profundidad a la que se descubre un estado es mínima;
- con costo uniforme, debe prevalecer el menor costo acumulado;
- con A^* y heurística consistente, un estado retirado con prioridad mínima ya tiene su costo óptimo;
- con una heurística admisible pero inconsistente, puede ser necesario **reabrir** un estado explorado si se encuentra un g menor.

La igualdad entre estados debe corresponder a la semántica del problema. En un recorrido con ventanas horarias, «estar en la zona X a las 9:00» y «estar en X a las 11:00» no son necesariamente el mismo estado. Si la hora se omite de la clave, el algoritmo puede podar una solución válida. En cambio, guardar detalles irrelevantes como un identificador accidental puede impedir reconocer duplicados y aumentar drásticamente el espacio.

7.1.5. Ciclos y estados repetidos

Un **ciclo** es un camino no vacío que comienza y termina en el mismo estado. Un estado repetido también puede surgir sin ciclos: dos ramas distintas pueden converger. Ambos fenómenos exigen distinguir tres controles:

1. **Control de ancestros:** rechaza un sucesor si su estado aparece en el camino actual. Evita ciclos dentro de una rama y requiere memoria proporcional a su profundidad, pero no detecta convergencias entre ramas.
2. **Conjunto de descubiertos:** registra estados al insertarlos en la frontera. Evita múltiples copias, apropiado cuando la primera llegada ya es la mejor según la estrategia.
3. **Mejor costo y reapertura:** conserva el menor g por estado. Si aparece un camino mejor, actualiza la frontera o reabre el estado. Es la opción general para búsquedas por costo.

Los ciclos de costo cero merecen atención: aunque no reduzcan el costo, pueden generar infinitas secuencias equivalentes en búsqueda en árbol. Los ciclos de

costo negativo son aún más problemáticos. Si un ciclo reduce el costo cada vez que se recorre, no existe un camino óptimo finito bien definido; costo uniforme y A* no ofrecen sus garantías habituales. En este capítulo se supondrán costos no negativos y, cuando se necesite una cota de terminación, un costo mínimo positivo ε .

Error habitual. Usar una lista de objetos nodo como conjunto explorado no detecta estados repetidos si cada objeto tiene identidad distinta. Debe utilizarse una clave canónica del estado. Otro error es cerrar un estado para siempre en A* sin haber demostrado consistencia de la heurística.

7.1.6. Complejidad temporal y espacial

El tamaño explícito de un grafo se expresa mediante $|V|$ y $|E|$. Si el algoritmo recorre cada vértice y arista una vez, su tiempo puede ser $O(|V| + |E|)$. En espacios implícitos, es más informativo usar:

- b : factor de ramificación máximo o efectivo;
- d : profundidad de la solución menos profunda;
- m : profundidad máxima del espacio, posiblemente infinita;
- C^* : costo de una solución óptima;
- ε : costo mínimo positivo de una acción.

Un árbol completo de ramificación b contiene $1 + b + \dots + b^d = O(b^d)$ nodos hasta profundidad d . La exponencial domina: reducir el factor de ramificación efectivo suele ser más valioso que acelerar una operación constante. Si $b = 10$, pasar de profundidad 5 a 6 multiplica aproximadamente por diez el último nivel.

La **complejidad temporal** puede medirse por nodos expandidos, nodos generados y costo de administrar la frontera. La **complejidad espacial** incluye frontera, explorados, nodos con punteros y representación de estados. Decir que una búsqueda «es rápida» sin declarar instancia, representación y métrica carece de valor comparativo.

Estructura o situación	Tiempo típico	Espacio típico	Observación
Recorrido explícito de grafo	$O(V + E)$	$O(V)$	Supone listas de adyacencia y control de visitados.
Árbol hasta profundidad d	$O(b^d)$	depende de la estrategia	La mayoría de nodos está en el nivel más profundo.
Cola de prioridad con N operaciones	$O(N \log N)$	$O(N)$	Se suma al costo de generar y evaluar estados.

Estructura o situación	Tiempo típico	Espacio típico	Observación
Estado con representación de tamaño k	al menos $O(k)$ por copia o clave	hasta $O(Nk)$	Una mala codificación puede dominar el cálculo.

Las cotas de peor caso no predicen por sí solas un caso concreto. Una heurística puede reducir enormemente el número de expansiones sin cambiar la clase exponencial. Por eso se combinan garantía teórica y evidencia experimental.

7.1.7. Ejemplo práctico guiado: representación de una red vial como grafo

Supóngase una red vial simplificada con intersecciones A, B, C, D, E, F . Las aristas dirigidas y sus tiempos estimados en minutos son: $A \rightarrow B : 4$, $A \rightarrow C : 2$, $C \rightarrow B : 1$, $B \rightarrow D : 5$, $C \rightarrow E : 7$, $B \rightarrow E : 2$, $E \rightarrow D : 1$, $D \rightarrow F : 3$ y $E \rightarrow F : 6$. El objetivo es ir de A a F minimizando tiempo.

La formulación es:

- **estado:** intersección actual;
- **estado inicial:** A ;
- **acciones:** recorrer una calle saliente habilitada;
- **transición:** llegar a la intersección destino de esa calle;
- **objetivo:** estado igual a F ;
- **costo:** tiempo de la arista; el costo del camino es su suma.

Existen, entre otros, los caminos $A - B - D - F$ de costo 12, $A - B - E - F$ de costo 12, $A - C - E - D - F$ de costo 13 y $A - C - B - E - D - F$ de costo 9. El camino con menos aristas no es el más barato. Una búsqueda en anchura podría devolver $A - B - D - F$; una búsqueda de costo uniforme debe devolver el camino de costo 9.

La representación es correcta solo para la pregunta planteada. Si se exige respetar combustible, calles cerradas por horario o una visita obligatoria, «intersección actual» deja de ser un estado suficiente. Habría que incorporar combustible restante, tiempo o tareas cumplidas. Asimismo, un tiempo estimado fijo abstrae congestión e incertidumbre: el resultado es óptimo en el modelo, no una promesa sobre el viaje real.

Comprobación guiada. Antes de buscar, conviene verificar: todas las aristas tienen dirección correcta; cada peso usa la misma unidad; no hay costos negativos; el objetivo es alcanzable desde el origen; y el estado contiene la información necesaria para decidir legalidad y costo futuro. Este pequeño ejemplo será la base conceptual para comparar estrategias, pero no debe confundirse con MOV-02: aquella actividad utiliza un grafo agregado de zonas y viajes observados, no calles.

7.2. Búsqueda no informada

Una estrategia **no informada** emplea la definición del problema y el costo recorrido, pero no una estimación específica de la cercanía al objetivo. «No informada» no significa aleatoria: cada algoritmo impone un orden preciso.

Sus garantías permiten establecer una línea base y entender qué aporta posteriormente una heurística.

7.2.1. Búsqueda en anchura

La **búsqueda en anchura** (BFS) expande primero todos los nodos de profundidad 0, luego los de profundidad 1 y así sucesivamente. Su frontera es una cola FIFO. En búsqueda en grafo, el estado suele marcarse al generarlo para evitar varias copias en la cola.

ANCHURA(problema):

```
frontera <- cola FIFO con la raiz
descubiertos <- {estado inicial}
mientras frontera no este vacia:
    nodo <- DESENCOLAR(frontera)
    si nodo satisface el objetivo: devolver su camino
    para cada sucesor de nodo:
        si su estado no esta en descubiertos:
            agregar estado a descubiertos
            ENCOLAR(frontera, sucesor)
devolver fracaso
```

Si b es finito y existe una solución a profundidad finita, BFS es completa. Es óptima cuando todos los pasos tienen igual costo, porque la primera solución extraída tiene longitud mínima. No es óptima con pesos diferentes. Su tiempo y memoria son $O(b^d)$: necesita conservar casi todo el último nivel. Esta exigencia de memoria suele ser su limitación práctica principal.

Un error común consiste en detenerse al generar cualquier objetivo sin analizar la estrategia. En BFS con costos unitarios es seguro porque los nodos se generan por niveles; en búsquedas de prioridad, generar un objetivo no demuestra que no exista otro camino pendiente más barato.

7.2.2. Búsqueda en profundidad

La **búsqueda en profundidad** (DFS) expande el nodo más profundo disponible. Puede implementarse con una pila LIFO o mediante recursión conceptual. Explora una rama hasta terminarla y luego retrocede.

Su atractivo es espacial: en búsqueda en árbol, si se guardan la rama actual y los hermanos pendientes, requiere $O(bm)$ memoria, muy inferior a la de anchura. Su tiempo de peor caso es $O(b^m)$. Si m es infinito o existen ciclos sin control, puede descender indefinidamente aun cuando haya una solución cercana en otra rama. En un espacio finito con conjunto explorado termina, pero no garantiza el camino más corto ni el de menor costo.

El orden de sucesores tiene gran influencia. Si desde el origen se insertan primero acciones hacia una región improductiva, DFS puede recorrerla completa antes de probar una solución inmediata. Por ello, comparar DFS sin fijar orden de vecinos produce resultados no reproducibles. DFS es útil cuando la memoria es crítica, las soluciones pueden estar muy profundas y cualquier solución es aceptable; no debe elegirse solo porque su implementación parece sencilla.

7.2.3. Búsqueda con profundidad limitada

La **búsqueda con profundidad limitada** (DLS) impide expandir nodos más allá de un límite ℓ . Convierte un espacio de profundidad infinita en uno finito y evita que una rama consuma toda la búsqueda. Debe distinguir tres resultados: solución, fracaso verdadero y **corte**, que significa que el límite impidió saber si existe solución más abajo.

```
PROFUNDIDAD-LIMITADA(nodo, limite):  
    si nodo satisface el objetivo: devolver solucion  
    si limite = 0: devolver corte  
    ocurrio_corte <- falso  
    para cada sucesor que no repita un ancestro:  
        resultado <- PROFUNDIDAD-LIMITADA(sucesor, limite - 1)  
        si resultado es solucion: devolver resultado  
        si resultado es corte: ocurrio_corte <- verdadero  
    si ocurrio_corte: devolver corte  
    devolver fracaso
```

El tiempo es $O(b^\ell)$ y el espacio $O(b\ell)$. DLS es completa solo si se elige $\ell \geq d$ y el espacio relevante cumple los supuestos de finitud. No es óptima en general. El desafío es fijar el límite: demasiado pequeño excluye todas las soluciones; demasiado grande recupera los problemas de DFS. Un conocimiento real del dominio, como un máximo operativo de trasbordos, puede justificarlo. Un número arbitrario no.

7.2.4. Profundización iterativa

La **profundización iterativa** (IDS) ejecuta DLS con límites $0, 1, 2, \dots$ hasta encontrar solución. Parece ineficiente porque regenera niveles superiores, pero en un árbol exponencial la mayor parte de los nodos está en el nivel final. Con $b = 10$ y $d = 5$, los pocos nodos cercanos a la raíz pueden repetirse varias veces sin dominar el total.

```
PROFUNDIZACION-ITERATIVA(problema):  
    para limite <- 0, 1, 2, ...:  
        resultado <- PROFUNDIDAD-LIMITADA(raiz, limite)  
        si resultado es solucion: devolver resultado  
        si resultado es fracaso: devolver fracaso
```

IDS combina completitud de BFS para factor de ramificación finito con memoria $O(bd)$ semejante a DFS. Es óptima cuando los costos son unitarios, pues encuentra primero una solución de profundidad mínima. Su tiempo es $O(b^d)$. Resulta especialmente adecuada cuando se desconoce d y almacenar la frontera de BFS es inviable. Si generar un estado es extraordinariamente caro o si se dispone de mucha memoria, la regeneración puede desaconsejarla.

7.2.5. Búsqueda de costo uniforme

La **búsqueda de costo uniforme** (UCS) extrae de una cola de prioridad el nodo con menor $g(n)$. Es una generalización de BFS: si cada paso cuesta una unidad, el costo coincide con la profundidad. Con costos distintos, UCS explora contornos de costo acumulado creciente.

COSTO-UNIFORME(problema):

```
frontera <- cola de prioridad por g con la raiz
mejor_g[estado inicial] <- 0
mientras frontera no este vacia:
    nodo <- EXTRAER-MINIMO(frontera)
    si nodo.g no coincide con mejor_g[nodo.estado]: continuar
    si nodo satisface el objetivo: devolver su camino
    para cada sucesor:
        nuevo_g <- nodo.g + costo del paso
        si estado no visto o nuevo_g < mejor_g[estado]:
            mejor_g[estado] <- nuevo_g
            insertar sucesor con prioridad nuevo_g
devolver fracaso
```

Con factor de ramificación finito y costo de paso $c \geq \varepsilon > 0$, UCS es completa y óptima. Antes de extraer una solución de costo C^* solo puede expandir nodos con $g < C^*$ y quizá algunos con $g = C^*$ según los empates. Una cota habitual expresa tiempo y espacio como

$$O(b^{1+\lceil C^*/\varepsilon \rceil}),$$

que puede ser peor que una cota por profundidad cuando existen pasos muy baratos. En grafos explícitos con pesos no negativos, UCS corresponde conceptualmente al algoritmo de Dijkstra desde un origen hasta que se extrae el objetivo.

No se debe marcar irrevocablemente un estado al generarlo: el primer camino descubierto puede no ser el más barato. Tampoco se debe aceptar un objetivo cuando se inserta; la optimalidad se establece cuando se extrae como mínimo. Costos negativos invalidan este razonamiento.

7.2.6. Completitud y optimalidad

Una estrategia es **completa** si garantiza encontrar una solución cuando existe, bajo los supuestos declarados. Es **óptima** si garantiza devolver una solución de costo mínimo. Son propiedades diferentes: DFS puede encontrar alguna solución sin hallar la mejor; una estrategia teóricamente óptima puede no terminar si se violan sus condiciones.

Las garantías siempre llevan supuestos. BFS requiere ramificación finita para no tener infinitos hijos antes del siguiente nivel. UCS requiere una cota positiva de costo para evitar infinitos pasos de costo decreciente antes de alcanzar C^* . A* necesita condiciones sobre la heurística. En un grafo desconectado, ningún algoritmo crea una ruta inexistente; completitud significa que reportará fracaso tras agotar el componente finito alcanzable.

También hay que precisar el objetivo de optimización. BFS es «óptima» respecto del número de pasos, no respecto de tiempo, distancia o dinero si esos costos difieren. Una suma ponderada de tiempo y distancia define un nuevo costo; ser óptimo para esa suma no significa ser simultáneamente óptimo para cada componente.

Lista de verificación de una afirmación de optimalidad: ¿qué costo se minimiza?, ¿los pesos tienen unidades coherentes?, ¿son no negativos?, ¿cuándo

se comprueba el objetivo?, ¿cómo se tratan estados repetidos?, ¿qué propiedad tiene la heurística?, ¿la solución se refiere al modelo o al mundo real?

7.2.7. Comparación de estrategias

La tabla resume cotas clásicas de búsqueda en árbol. En búsqueda en grafo finito, las cotas pueden expresarse mediante $|V|$ y $|E|$, pero la frontera y la gestión de prioridades siguen influyendo.

Estrategia	Orden de extracción	Completa	Óptima	Tiempo de peor caso	Espacio de peor caso
Anchura	menor profundidad FIFO	Sí, si b es finito	Sí, con costos unitarios	$O(b^d)$	$O(b^d)$
Profundidad mayor	profundidad mayor LIFO	No en espacios infinitos	No	$O(b^m)$	$O(bm)$
Profundidad limitada	profundidad mayor hasta ℓ	Solo si $\ell \geq d$	No	$O(b^\ell)$	$O(b\ell)$
Profundización iterativa	límites crecientes	Sí, si b es finito	Sí, con costos unitarios	$O(b^d)$	$O(bd)$
Costo uniforme	menor g	Sí, si $c \geq \varepsilon > 0$	Sí, con costos no negativos	exponencial en C^*/ε	igual orden que tiempo

La elección no se reduce a «mejor algoritmo». Si todos los pasos cuestan lo mismo y la solución es poco profunda, BFS ofrece una referencia clara. Si la memoria domina y una solución cualquiera sirve, DFS o DLS pueden ser razonables. Si se desconoce la profundidad, IDS evita escoger un límite arbitrario. Si los costos difieren y se exige mínimo costo, UCS es la línea base correcta.

En una comparación experimental justa se mantienen constantes el grafo, origen, objetivo, función de costo, criterio de objetivo y orden de sucesores. Se informan camino, costo, profundidad, nodos generados, nodos expandidos, máximo de frontera y tiempo bajo el mismo entorno. Comparar solo milisegundos mezcla estrategia, implementación y carga del sistema.

7.2.8. Ejemplo práctico guiado: comparación experimental de algoritmos no informados

Considérese el grafo dirigido con sucesores en el orden mostrado:

Estado	Sucesores y costos
S	$A(1), B(4), C(2)$
A	$D(8), E(2)$
B	$E(1), G(7)$
C	$B(1), F(3)$
D	$G(1)$

Estado	Sucesores y costos
E	$G(3)$
F	$G(2)$

El objetivo es G . BFS organiza por profundidad. Tras S , genera A, B, C ; en el nivel siguiente aparecen candidatos a través de A, B y C . El primer objetivo generado desde B forma $S - B - G$, de dos pasos y costo 11. Según se compruebe el objetivo al generar o al extraer, cambian algunos conteos, pero no la solución mínima en pasos.

DFS sigue primero $S - A - D - G$ con el orden indicado y devuelve costo $1 + 8 + 1 = 10$. Ha encontrado rápidamente una solución, pero no la mejor. DLS con $\ell = 2$ encuentra $S - B - G$; con $\ell = 1$ devuelve corte. IDS repite raíz y primer nivel antes de hallar también una solución de dos pasos.

UCS compara costos acumulados. Extrae inicialmente $S(0)$, $A(1)$, $C(2)$, $E(3)$ y $B(3)$; la llegada $S - C - B$ mejora el costo inicial de $B(4)$. Luego aparecen $F(5)$ y alternativas a E . El objetivo óptimo puede alcanzarse por $S - A - E - G$ con costo 6 o por $S - C - F - G$ con costo 7; UCS extrae primero la solución de costo 6. El ejemplo demuestra tres hechos: menor profundidad no implica menor costo; una llegada posterior puede mejorar un estado; y la estrategia cambia tanto la solución como la memoria.

Un informe experimental completaría una tabla con los conteos reales de una convención fija. Antes de interpretar diferencias pequeñas, debería repetir el ensayo, explicar empates y verificar automáticamente que el costo reconstruido coincide con la suma de aristas. El propósito no es declarar un ganador universal, sino relacionar resultados con las garantías.

La búsqueda no informada ofrece referencias sólidas, pero ignora información obvia sobre el destino. La sección siguiente estudia cómo incorporar esa información sin confundir una estimación útil con una garantía.

7.3. Búsqueda informada y heurísticas

La búsqueda informada utiliza conocimiento adicional para ordenar la exploración. Una buena heurística dirige hacia regiones prometedoras; una mala puede añadir cálculo sin ahorrar expansiones o, si se emplea sin condiciones, comprometer la optimalidad. La pregunta central no es solo «¿qué valor estima?», sino «¿qué relación demostrable mantiene con el costo verdadero?».

7.3.1. Funciones heurísticas

Una **heurística** es una función $h : S \rightarrow \mathbb{R}$ que estima el costo mínimo restante desde un estado hasta un objetivo. Se adopta $h(g) = 0$ para estados objetivo. El costo verdadero restante se denota $h^*(n)$. Una heurística no modifica el costo real g ; aporta una señal para priorizar.

Ejemplos habituales son distancia en línea recta para rutas, cantidad de piezas fuera de lugar en un rompecabezas o suma de costos mínimos pendientes en una asignación. La escala debe coincidir con la función objetivo. Kilómetros no pueden sumarse directamente a minutos; si el costo es tiempo, una distancia puede transformarse en una cota temporal mediante una velocidad máxima válida.

Dos criterios empíricos son el costo de calcular h y su capacidad de discriminación. Una heurística perfecta, $h = h^*$, conduce directamente a una solución óptima, pero calcularla suele equivaler a resolver el problema. La heurística nula $h = 0$ no cuesta casi nada y convierte A^* en UCS. El diseño busca una aproximación barata y fundamentada.

7.3.2. Búsqueda voraz

La **búsqueda voraz por el mejor primero** ordena la frontera solo por $h(n)$. Elige el estado que parece más cercano al objetivo e ignora cuánto costó alcanzarlo. Puede avanzar con pocas expansiones cuando la estimación orienta bien, pero puede aceptar un desvío inicial muy caro o quedar atraída por una región engañosa.

En un grafo con dos opciones desde S , supóngase $S \rightarrow A$ de costo 100 con $h(A) = 1$ y $S \rightarrow B$ de costo 1 con $h(B) = 5$. Si desde A se llega al objetivo por costo 1 y desde B por costo 5, la voraz elige $S - A - G$ de costo 101, aunque existe $S - B - G$ de costo 6. La heurística describe cercanía restante, no costo total.

Con control de repetidos y un grafo finito, la búsqueda voraz termina; en espacios infinitos no es completa en general. Tampoco es óptima, incluso con una heurística admisible. Su utilidad está en obtener rápidamente soluciones plausibles cuando la optimalidad no es obligatoria o como componente de procedimientos más complejos. Debe evaluarse la calidad de solución, no solo la velocidad.

7.3.3. Algoritmo A^*

A^* combina costo recorrido y costo estimado:

$$f(n) = g(n) + h(n).$$

Interpreta f como una estimación del costo total de una solución que pasa por n . Extrae de la frontera el nodo con menor f . Su pseudocódigo es el de costo uniforme reemplazando la prioridad g por $g + h$, pero conserva mejor_g para decidir si una ruta nueva mejora el estado.

```
A-ESTRELLA(problema, h):
  frontera <- cola de prioridad por f
  mejor_g[inicial] <- 0
  insertar raiz con f = h(inicial)
  mientras frontera no este vacia:
    nodo <- EXTRAER-MINIMO(frontera)
    si nodo.g es una entrada obsoleta: continuar
    si nodo satisface el objetivo: devolver su camino
    para cada sucesor:
      nuevo_g <- nodo.g + costo_paso
      si nuevo_g mejora mejor_g[estado sucesor]:
        registrar padre y nuevo_g
        insertar con f = nuevo_g + h(sucesor)
  devolver fracaso
```

Lectura tipada del pseudocódigo El pseudocódigo combina conceptos del problema con estructuras de datos de una implementación. Distinguirlos evita interpretar cada nombre como si fuera un estado o una variable simple.

Término	Qué representa	Tipo conceptual
problema	Definición del espacio de estados, estado inicial, objetivo, sucesores y costos	registro u objeto del problema
h	Función que estima el costo restante desde un estado hasta un objetivo	función estado -> número real
frontera	Nodos pendientes de expansión, ordenados por prioridad	cola de prioridad mínima
mejor_g	Menor costo acumulado conocido para cada estado	diccionario estado -> número real
inicial	Estado desde el que comienza la búsqueda	estado
raíz	Nodo de búsqueda creado para inicial	registro u objeto nodo
nodo	Camino parcial almacenado junto con su estado y costos	registro u objeto nodo
f	Prioridad de A*, calculada como $g+h$	número real

Un **estado** describe una configuración del problema, como una ciudad o una posición. Un **nodo de búsqueda** es un registro creado por el algoritmo: puede contener ese estado, un puntero a su padre, la acción que lo produjo, g, h y f. Dos nodos pueden referirse al mismo estado y tener distintos padres o valores de g porque fueron alcanzados por caminos diferentes.

En la inicialización, `mejor_g[inicial] <- 0` significa que el costo conocido desde el estado inicial hasta sí mismo es cero. La raíz se inserta con `f=h(inicial)` porque su g vale cero y, por tanto, `f=g+h=0+h(inicial)`.

EXTRAER-MINIMO(frontera) retira la entrada cuya prioridad f es menor. Si varias entradas empatan, debe existir una regla estable y documentada, como el orden de inserción. Extraer no significa todavía expandir: primero se verifica si la entrada sigue representando el mejor camino conocido.

Una **entrada obsoleta** es un nodo cuyo costo acumulado es peor que el que figura actualmente en `mejor_g`:

```
nodo.g > mejor_g[nodo.estado]
```

Esto ocurre cuando se conserva una entrada vieja en la cola y posteriormente se encuentra un camino más barato. Una implementación puede actualizar la cola en el lugar o insertar una nueva entrada y descartar la vieja al extraerla.

Un **sucesor** se representa conceptualmente como:

```
(acción, estado_siguiete, costo_paso)
```

La **acción** es la operación disponible desde el estado actual; el estado siguiente es el resultado de aplicarla; y `costo_paso` es el costo numérico de esa transición. En

un grafo de rutas, $(ir_a_A, A, 2)$ expresa que se ejecuta la acción de ir a A, se llega al estado A y la transición cuesta 2.

nodo.g es el costo acumulado desde el estado inicial hasta el estado del nodo. costo_paso es el costo de la transición que lleva desde ese nodo hasta su sucesor. Ambos deben ser números no negativos y usar la misma unidad. Por eso:

```
nuevo_g <- nodo.g + costo_paso
```

calcula el costo acumulado del nuevo camino. Si nodo.g=4 y costo_paso=3, entonces nuevo_g=7.

La condición `nuevo_g mejora mejor_g[estado sucesor]` equivale a comprobar que el sucesor todavía no tiene un costo registrado o que nuevo_g es menor que el anterior. Solo en ese caso se actualiza mejor_g, se guarda el padre y se inserta una nueva entrada en la frontera con:

```
f = nuevo_g + h(estado_sucesor)
```

La prueba de objetivo se realiza al extraer el nodo, no simplemente al generarlo. devolver su camino significa seguir los punteros padre desde el nodo objetivo hasta la raíz e invertir la secuencia. devolver fracaso es un resultado explícito que indica que la frontera se agotó sin encontrar un objetivo alcanzable.

Con $h = 0$, A^* es UCS. A medida que una heurística informativa aumenta sin sobreestimar, A^* suele expandir menos estados. Sin embargo, guarda una frontera potencialmente exponencial; su limitación habitual es la memoria. Los empates entre valores f pueden resolverse prefiriendo mayor g o menor h para avanzar más profundo entre candidatos equivalentes, pero la regla debe declararse.

A^* no significa automáticamente «óptimo». La garantía depende de costos no negativos, prueba de objetivo al extraer, tratamiento correcto de mejores caminos y propiedades de h . Una implementación que mezcla unidades o descarta reaperturas necesarias deja de corresponder al algoritmo analizado.

7.3.4. Heurísticas admisibles

Una heurística es **admisible** si nunca sobreestima el costo óptimo restante:

$$0 \leq h(n) \leq h^*(n) \quad \text{para todo } n.$$

Es una cota inferior, no una predicción promedio. Que «casi siempre acierte» no basta: una sola sobreestimación puede hacer que A^* evite el camino óptimo. En búsqueda en árbol, con costos no negativos y una heurística admisible, A^* devuelve una solución óptima. La intuición es que todo nodo de un camino óptimo tiene $f(n) = g(n) + h(n) \leq C^*$; una solución subóptima tiene $f = C > C^*$. Por tanto, esta última no debería extraerse antes de que la frontera permita completar una óptima.

La admisibilidad puede derivarse de un problema relajado. Si se eliminan restricciones, el costo óptimo relajado no puede superar el original y sirve como h . Para un desplazamiento cuyo costo es distancia recorrida, la distancia recta entre posiciones es una cota inferior si las aristas representan trayectos físicos y sus pesos no son menores que esa separación. Para tiempo, distancia recta dividida por una velocidad máxima físicamente y operacionalmente válida puede ser cota inferior.

Multiplicar una heurística admisible por un factor mayor que uno suele perder admisibilidad, aunque reduzca expansiones. Esa variante puede ser útil como búsqueda ponderada si se acepta una solución aproximada, pero no debe presentarse como A* óptimo sin una garantía específica.

7.3.5. Heurísticas consistentes

Una heurística es **consistente** o monótona si para toda transición $n \rightarrow n'$ de costo $c(n, n')$ cumple

$$h(n) \leq c(n, n') + h(n').$$

Es una desigualdad triangular: la estimación desde n no supera el costo de dar un paso más la estimación desde el sucesor. Si además $h(g) = 0$, la consistencia implica admisibilidad. A lo largo de cualquier camino,

$$f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n),$$

de modo que los valores f no disminuyen. En A* con búsqueda en grafo, esto permite cerrar un estado cuando se extrae con prioridad mínima: no aparecerá después un camino más barato.

Una heurística admisible puede ser inconsistente. En ese caso A* todavía puede ser óptimo si reabre estados cuyo g mejora. Prohibir reaperturas puede devolver una solución subóptima. La consistencia se verifica localmente sobre aristas, mientras que la admisibilidad se refiere al costo global desconocido; por eso la primera suele ser más fácil de auditar en un grafo finito.

Prueba práctica. Para cada arista almacenada, calcular la diferencia $h(u) - h(v)$ y comprobar que no excede $c(u, v)$, con tolerancia numérica documentada. Cero violaciones observadas en un grafo concreto demuestra consistencia en ese grafo si se revisaron todas sus aristas; no la demuestra para datos futuros o aristas omitidas.

7.3.6. Diseño y evaluación de heurísticas

Una heurística puede diseñarse mediante varios principios:

- **relajación:** resolver una versión con menos restricciones;
- **descomposición:** sumar costos mínimos de subproblemas independientes, evitando contar dos veces el mismo recurso;
- **abstracción:** buscar exactamente en un espacio más pequeño y usar ese costo;
- **geometría:** construir cotas a partir de distancias y límites físicos;
- **patrones precomputados:** almacenar costos exactos de configuraciones abstractas cuando la memoria lo permite.

Si h_1 y h_2 son admisibles, $\max(h_1, h_2)$ también lo es y domina a cada una: nunca es menor y sigue sin sobreestimar. La suma solo es admisible si los costos que estima son aditivos y no se superponen. Elegir el mínimo conserva admisibilidad, pero produce una guía más débil.

La evaluación debe incluir propiedades y desempeño. Primero se verifica unidad, objetivo cero, no negatividad, admisibilidad cuando sea posible y consistencia arista por arista. Después se compara con $h = 0$ sobre las mismas instancias. Métricas útiles son nodos expandidos, generados, reaperturas, máximo de frontera, tiempo de evaluación de h , costo de solución y, si se conoce h^* , error $h^* - h$. El **factor de ramificación efectivo** b^* puede estimarse resolviendo $N + 1 = 1 + b^* + \dots + (b^*)^d$ para N expansiones; valores menores indican mejor orientación.

Una correlación alta entre h y costo real no demuestra admisibilidad. Tampoco basta probar pocos pares cómodos. Deben incluirse distancias cortas y largas, componentes poco conectados, empates y casos sin solución. Si la heurística depende de datos estimados, se documentan periodo, filtros y posibles cambios.

7.3.7. Ejemplo práctico guiado: búsqueda de rutas con A*

Retómese una red física simplificada. Cada nodo tiene coordenadas y cada arista dirigida pesa su longitud de recorrido. Se define $h(n)$ como distancia euclidiana desde n al objetivo. Si cada longitud de arista es al menos la distancia euclidiana entre sus extremos, la desigualdad triangular garantiza consistencia:

$$h(n) \leq d(n, n') + h(n') \leq c(n, n') + h(n').$$

Supónganse los costos y heurísticas hacia G : desde S salen A con costo 2 y $h(A) = 4$, y B con costo 2 y $h(B) = 7$. A* asigna $f(A) = 6$ y $f(B) = 9$, por lo que explora primero A . Desde A llega a C con costo adicional 2 y $h(C) = 2$, luego $f(C) = 6$. Si $C \rightarrow G$ cuesta 3, alcanza una solución de costo 7. UCS habría considerado también varios nodos de costo acumulado inferior a 7 aunque se alejaran del objetivo. A* evita algunos porque su cota total ya es alta.

El procedimiento guiado es:

1. fijar origen, objetivo y costo de aristas;
2. proponer una heurística en la misma unidad;
3. justificar que es cota inferior y comprobar consistencia en todas las aristas;
4. ejecutar UCS y A* con idénticas reglas de repetición y desempate;
5. confirmar que ambos devuelven el mismo costo óptimo;
6. comparar expansiones y costo de calcular la heurística;
7. repetir sobre varios pares, incluidos casos donde la ventaja sea pequeña.

Si el peso cambia de distancia a tiempo observado, la distancia euclidiana ya no está en la misma unidad. Puede dividirse por una velocidad máxima documentada. Usar velocidad media sería riesgoso: un trayecto real puede ser más rápido que la media y la estimación sobreestimaría su tiempo. Además, coordenadas de centroides describen zonas, no puntos exactos de inicio y fin. Estas limitaciones serán centrales en la actividad aplicada.

Actividad EMO [MOV-02]: comparar rutas y heurísticas sobre una red

Capacidad mínima: representar una red de movilidad como grafo y justificar una estrategia de búsqueda.

Pregunta realizable: ¿qué caminos entre zonas pueden construirse sobre la red dirigida de conexiones origen-destino observadas en los viajes de taxi amarillo y cómo cambia la búsqueda según el costo?

Esta actividad se alinea con el Apéndice D. Su red **no es una red vial**: los nodos son zonas TLC y una arista $i \rightarrow j$ indica que la instantánea contiene suficiente cantidad de viajes válidos con ascenso en i y descenso en j . El camino $i \rightarrow j \rightarrow k$ significa que existen conexiones OD observadas entre esos pares; no significa que un vehículo haya realizado ese itinerario, que las zonas sean contiguas ni que la ruta cruce determinadas calles. Una recomendación calle por calle exigiría una red vial externa, fuera del caso mínimo.

Fuente y unidad. Se utilizan los archivos NYC TLC Yellow Taxi indicados en el Apéndice D para febrero, marzo y abril de 2026, junto con Taxi Zone Lookup y, si se construye una heurística geográfica, Taxi Zone Shapefile. La unidad original es un viaje reportado. El nivel derivado para MOV-02 es el grafo OD.

Variables requeridas: PULocationID, DOLocationID, tpep_pickup_datetime, tpep_dropoff_datetime y trip_distance. Antes de agregar, se valida que el pickup pertenezca al periodo declarado, que el drop-off sea posterior, que duración y distancia sean plausibles en conjunto y que los identificadores existan en el lookup o queden marcados como desconocidos. Las exclusiones deben conservar conteos; no se eliminan registros silenciosamente.

Construcción formal del grafo. Sea Z el conjunto de zonas TLC válidas. Para cada par ordenado (i, j) se calcula el soporte q_{ij} , cantidad de viajes válidos observados. Dado un umbral mínimo $q_{\min}$ documentado,

$$V = Z, \quad E = \{(i, j) : q_{ij} \geq q_{\min}\}.$$

El umbral evita que una arista dependa de un único viaje y reduce conexiones inestables. No debe elegirse solo para obtener un camino deseado. Se informa cómo cambian cantidad de aristas, zonas aisladas y conectividad al variar razonablemente $q_{\min}$.

Cada arista conserva al menos soporte, duración mediana y distancia mediana. La mediana reduce la influencia de valores extremos, pero no elimina la necesidad de control de calidad. El costo elegido debe responder a una pregunta:

- $c_{ij} = \text{mediana}(\text{duracion}_{ij})$ para minimizar tiempo típico agregado;
- $c_{ij} = \text{mediana}(\text{distancia}_{ij})$ para minimizar distancia reportada típica;
- una combinación normalizada y justificada si existe una decisión multicriterio.

No se mezclan minutos y millas mediante una suma sin escala e interpretación. Tampoco se usa $1/q_{ij}$ como si fuera distancia: podría representar preferencia por conexiones frecuentes, pero resolvería otra pregunta. El costo agregado no garantiza el tiempo o distancia de un viaje futuro y la suma de medianas de varias aristas es un criterio analítico, no un itinerario observado completo.

Selección de instancias. El equipo propone varios pares origen-destino dentro del mismo componente alcanzable; cada estudiante analiza al menos un par asignado. Debe incluirse un par no trivial cuya solución tenga más de una arista y, cuando sea posible, un caso con alternativas de costos distintos. Si no hay camino dirigido, el resultado correcto es «inalcanzable en el grafo filtrado», no invertir aristas ni reducir el umbral después de ver el resultado sin reportarlo.

Comparación obligatoria: costo uniforme y A* se ejecutan sobre exactamente el mismo grafo, par OD, costo, control de repetidos y regla de desempate. Se registran camino de zonas, costo total, cantidad de aristas, nodos generados, nodos expandidos, máximo de frontera y tiempo de cálculo. Ambos deben devolver el mismo costo si A* cumple sus condiciones; una diferencia obliga a revisar heurística, cierres, unidades o reconstrucción.

Heurística para costo de distancia. Si se dispone de geometría, puede calcularse una separación geográfica entre centroides de zonas. Para sostener admisibilidad debe verificarse que, para cada arista, la heurística respete $h(i) \leq c_{ij} + h(j)$. Un centroide es un representante agregado: la distancia entre centroides podría superar una distancia mediana reportada en ciertos pares, por lo que la frase «es línea recta» no constituye por sí sola una prueba. Una opción conservadora consiste en calibrar una escala $0 \leq \alpha \leq 1$ y definir $h(i) = \alpha d(i, g)$, eligiendo α sin usar la instancia de prueba y verificando consistencia en todo el grafo analizado. Si no puede justificarse una cota positiva, $h = 0$ es admisible y convierte A* en UCS.

Heurística para costo de tiempo. Una candidata es

$$h(i) = \frac{d(i, g)}{v_{\max}},$$

con unidades convertidas correctamente y una velocidad máxima documentada. $v_{\max}$ debe producir una cota inferior plausible, no ser la velocidad media de los viajes. Aun así, se comprueba la desigualdad de consistencia sobre todas las aristas porque centroides, agregación por medianas y errores de registro pueden quebrarla. Si existen violaciones, se reduce la escala, se usa reapertura con una discusión de admisibilidad o se adopta $h = 0$; no se ocultan las violaciones.

Protocolo de trabajo:

1. definir periodo, filtros y conteos de calidad;
2. agregar pares OD y fijar $q_{\min}$ antes de seleccionar resultados favorables;
3. describir $|V|$, $|E|$, dirección, componentes y distribución de soportes;
4. elegir una sola función de costo principal y declarar unidad;
5. seleccionar pares reproducibles mediante LocationID y nombre de zona;
6. obtener la referencia con costo uniforme;
7. diseñar la heurística usando solo información permitida;
8. evaluar objetivo cero, no negatividad y consistencia en todas las aristas;
9. ejecutar A* bajo las mismas condiciones;
10. interpretar diferencias y efectuar sensibilidad al umbral o a la escala heurística.

Modalidad de trabajo: construcción y auditoría del grafo en equipo; experimentos y análisis individuales. Compartir el grafo no implica compartir conclusiones: cada estudiante debe explicar su instancia, propiedades de la heurística y limitaciones.

Evidencia individual: notebook reproducible con representación del grafo, definición de estados, acciones, objetivo y costos; resumen de filtros y soporte; resultados comparativos; comprobaciones de la heurística; y discusión de admisibilidad o limitaciones. El capítulo no prescribe código de implementación: el notebook debe materializar los conceptos con las herramientas del curso.

Tabla mínima de resultados:

Campo	Costo uniforme	A*
Origen y destino	mismos	mismos
Costo y unidad	mismo	mismo
Camino de LocationID	reportar	reportar
Costo total	reportar	reportar
Nodos expandidos	reportar	reportar
Nodos generados	reportar	reportar
Máximo de frontera	reportar	reportar
Reaperturas	reportar	reportar

Criterios de aprobación:

- Estados, acciones, costos y objetivo quedan definidos sin ambigüedad.
- Los algoritmos se comparan sobre las mismas instancias.
- La heurística utiliza información disponible y sus propiedades se justifican.
- Las aristas exigen soporte mínimo y preservan su dirección OD.
- Se distinguen distancia reportada, separación entre centroides y distancia vial.
- El producto se presenta como camino en una red de zonas conectadas por viajes observados, nunca como ruta calle por calle.
- Se reconoce que TLC cubre taxis amarillos reportados y no toda la movilidad de Nueva York.

Errores que invalidan la evidencia: interpretar zonas como intersecciones; conectar centroides por cercanía sin viajes OD; usar el shapefile como red vial; comparar algoritmos con diferentes filtros; afirmar admisibilidad porque la heurística «parece menor» en un solo camino; seleccionar una velocidad promedio como cota máxima; o concluir que menos expansiones implica mejor ruta cuando ambos algoritmos deben coincidir en costo óptimo.

Aporte al laboratorio: incorpora la dimensión de decisión o desplazamiento sobre la estructura espacial. Su conectividad puede alimentar posteriormente una regla didáctica de prioridad de zonas, pero no constituye despacho óptimo: los datos no contienen ubicación ni disponibilidad de vehículos libres, demanda insatisfecha ni otros modos de transporte.

7.4. Búsqueda local y problemas con restricciones

Los algoritmos anteriores construyen caminos desde un estado inicial. En muchos problemas de optimización importa la configuración final y no la secuencia utilizada para obtenerla. La búsqueda local mantiene una o pocas soluciones completas y se mueve entre vecinas, con memoria reducida. Por otra parte, cuando el objetivo principal es satisfacer reglas combinatorias, una formulación de satisfacción de restricciones permite explotar la estructura de variables y dominios.

7.4.1. Búsqueda por ascenso de colinas

El **ascenso de colinas** parte de una solución candidata y reemplaza el estado actual por un vecino mejor según una función de valor. Si se minimiza un costo J , «mejor» significa menor J ; el nombre ascenso supone convencionalmente que se maximiza $F = -J$. No mantiene un camino hacia el origen ni una frontera global.

```

ASCENSO-DE-COLINAS(estado_inicial):
    actual <- estado_inicial
    repetir:
        vecinos <- VECINOS(actual)
        si vecinos esta vacio: devolver actual
        siguiente <- un vecino de mejor valor
        si VALOR(siguiente) <= VALOR(actual): devolver actual
        actual <- siguiente

```

En un recorrido, un estado puede ser una permutación de visitas y un vecino puede intercambiar dos posiciones o invertir un segmento. La calidad del vecindario determina qué mejoras son accesibles en un paso. Un vecindario muy pequeño produce muchos óptimos locales; uno enorme encarece cada iteración.

El método usa poca memoria y suele obtener soluciones aceptables rápidamente en espacios grandes. No es completo ni garantiza óptimo global. Variantes útiles incluyen elegir la primera mejora en lugar de examinar todos los vecinos, permitir movimientos laterales limitados y ejecutar reinicios desde estados aleatorios. Los reinicios no prueban optimalidad, pero permiten estimar estabilidad: si muchas ejecuciones convergen al mismo valor, aumenta la confianza empírica.

7.4.2. Máximos locales, mesetas y crestas

Un **máximo local** tiene valor no menor que todos sus vecinos, aunque exista otro estado lejano con valor superior. Una **meseta** es una región de igual valor sin dirección local clara. Una **cresta** presenta una dirección de mejora que no puede seguirse con los movimientos disponibles, por ejemplo cuando requiere cambiar simultáneamente varias decisiones.

Las respuestas posibles tienen costos distintos:

- reinicios aleatorios exploran otras cuencas de atracción;
- movimientos laterales atraviesan mesetas, pero deben limitarse para evitar ciclos;
- vecindarios variables alternan tipos de movimiento;
- búsqueda tabú recuerda movimientos recientes y evita regresar inmediatamente;
- perturbaciones ocasionales abandonan una región antes de volver a mejorar.

El diagnóstico exige observar trayectorias, no solo el resultado final. Una curva completamente plana puede indicar meseta, pero también un error en la función objetivo. Resultados idénticos en todos los reinicios pueden señalar un óptimo dominante o una inicialización no realmente variable. Toda comparación debe fijar presupuesto de evaluaciones, no solo cantidad de iteraciones, porque examinar un vecindario grande cuesta más.

Limitación conceptual. La búsqueda local puede visitar temporalmente soluciones inviables si la representación lo permite. Penalizar violaciones mediante $J(x) = costo(x) + \lambda violaciones(x)$ no garantiza factibilidad: un λ pequeño tolera infracciones y uno enorme aplanar diferencias entre soluciones inviables. Cuando las restricciones son duras, es preferible generar solo vecinos factibles, reparar candidatos o usar un método específico de restricciones.

7.4.3. Recocido simulado

El **recocido simulado** incorpora la posibilidad de aceptar empeoramientos para escapar de óptimos locales. Al minimizar energía J , un movimiento con $\Delta = J(\text{nuevo}) - J(\text{actual}) \leq 0$ se acepta siempre. Si $\Delta > 0$, se acepta con probabilidad

$$p = \exp(-\Delta/T),$$

donde $T > 0$ es la temperatura. A temperatura alta se aceptan diversos movimientos; al enfriarse, el comportamiento se aproxima al ascenso de colinas.

```
RECOCIDO-SIMULADO(estado_inicial, calendario):
    actual <- estado_inicial
    mejor <- actual
    para t <- 1 hasta presupuesto:
        T <- calendario(t)
        si T <= 0: devolver mejor
        nuevo <- vecino elegido al azar de actual
        delta <- COSTO(nuevo) - COSTO(actual)
        si delta <= 0 o AZAR(0,1) < exp(-delta / T):
            actual <- nuevo
        si COSTO(actual) < COSTO(mejor): mejor <- actual
    devolver mejor
```

Debe devolverse el mejor estado observado, no necesariamente el último. La escala de T debe ser compatible con la de Δ ; cambiar unidades del costo sin ajustar temperatura altera el algoritmo. Un calendario geométrico usa $T_t = T_0 \alpha^t$, con $0 < \alpha < 1$. Un enfriamiento muy rápido reproduce el atrapamiento del ascenso; uno muy lento consume presupuesto explorando casi al azar.

Bajo condiciones matemáticas de enfriamiento extremadamente lento, existen resultados de convergencia al óptimo global, pero rara vez son prácticos. En uso real el recocido es estocástico y aproximado. Se reportan semilla, calendario, presupuesto y distribución de resultados de varias ejecuciones. Comparar solo la mejor ejecución de un método con la mediana de otro es una práctica sesgada.

7.4.4. Introducción a algoritmos evolutivos

Los **algoritmos evolutivos** mantienen una población de candidatos. Cada candidato posee una representación o genotipo; una función de aptitud evalúa su calidad; la selección favorece candidatos prometedores; el cruce combina material de progenitores; y la mutación introduce variación. El reemplazo forma la siguiente generación. No se trata de una copia literal de la evolución biológica, sino de una familia de heurísticas de optimización.

```
ALGORITMO-EVOLUTIVO(poblacion_inicial):
    poblacion <- poblacion_inicial
    mientras no se cumpla el criterio de parada:
        evaluar aptitud de cada individuo
        padres <- seleccionar segun aptitud y diversidad
        descendencia <- cruzar y mutar padres
        reparar o descartar descendencia invalida
```

```
poblacion <- reemplazar con supervivientes y descendencia  
devolver el mejor individuo factible observado
```

La representación debe hacer probables las soluciones válidas. Para recorridos, un cruce ordinario de vectores puede duplicar visitas y omitir otras; se requieren operadores que preserven permutaciones o mecanismos de reparación. Una presión de selección excesiva elimina diversidad y causa convergencia prematura; una mutación demasiado alta transforma el proceso en búsqueda aleatoria.

Estos algoritmos son apropiados cuando el espacio es discontinuo, existen múltiples objetivos o no hay gradientes útiles. Consumen muchas evaluaciones y no garantizan el óptimo. Deben compararse bajo igual presupuesto, con varias semillas y baselines simples. Esta introducción aporta vocabulario y criterios; el diseño avanzado de operadores y optimización multiobjetivo excede el alcance del capítulo.

7.4.5. Variables, dominios y restricciones

Un **problema de satisfacción de restricciones** (CSP) se define como

$$\mathcal{C} = (X, D, C),$$

donde $X = \{X_1, \dots, X_n\}$ es un conjunto de variables; cada D_i es el dominio de valores posibles para X_i ; y C es un conjunto de restricciones. Una asignación es **parcial** si solo fija algunas variables, **completa** si fija todas, **consistente** si no viola restricciones aplicables y **solución** si es completa y satisface todas las restricciones.

Las restricciones pueden ser unarias, binarias o globales. **TODO-DIFERENTE**($X_1, \dots, X_k$) es una restricción global más expresiva que enumerar pares, aunque ambas formulaciones compartan soluciones. También se distinguen restricciones duras, que deben cumplirse, y preferencias blandas, cuya violación tiene costo. Mezclarlas sin indicarlo transforma un CSP en un problema de optimización con restricciones.

Ejemplo: asignar cuatro inspecciones $I_1, \dots, I_4$ a franjas mañana y tarde y a dos equipos. Cada variable puede representar el par (franja, equipo). Las restricciones exigen que un equipo no realice dos inspecciones simultáneas, que I_1 ocurra por la mañana y que I_3 sea posterior a I_2 . Si además se minimiza desplazamiento, la factibilidad es el primer nivel y el costo, el segundo.

El **grafo de restricciones** coloca variables como nodos y une aquellas que participan en una restricción binaria. Su estructura explica parte de la dificultad: componentes independientes pueden resolverse por separado; un árbol admite algoritmos eficientes; redes densas propagan más información, pero también acoplan decisiones.

7.4.6. Satisfacción de restricciones

La búsqueda básica para CSP es el **retroceso** o backtracking: asigna una variable, prueba un valor consistente y retrocede cuando ninguna continuación es posible. A diferencia de una búsqueda genérica, el orden de asignación de variables no cambia la solución final y puede explotarse con heurísticas especializadas.

```

RETROCESO(asignacion, dominios):
  si asignacion es completa: devolver asignacion
  X <- variable no asignada elegida por MRV y grado
  para cada valor v de D[X] ordenado por valor menos restrictivo:
    si v es consistente con asignacion:
      asignar X <- v
      reducciones <- PROPAGAR(X, v, dominios)
      si ningun dominio queda vacio:
        resultado <- RETROCESO(asignacion, dominios reducidos)
        si resultado no es fracaso: devolver resultado
      deshacer asignacion y reducciones
  devolver fracaso

```

La heurística **MRV** elige la variable con menos valores restantes: intenta fallar pronto. En empate, la heurística de **grado** elige la variable que restringe más variables no asignadas. El **valor menos restrictivo** prueba primero el que elimina menos opciones ajenas. Estas heurísticas ordenan la búsqueda; no relajan restricciones.

La comprobación hacia adelante elimina de dominios vecinos los valores incompatibles con la asignación recién hecha. La **consistencia de arco** exige, para cada valor de X , algún valor compatible en cada variable vecina Y . El procedimiento AC-3 revisa arcos hasta que no haya cambios o un dominio quede vacío. Propagar detecta fracasos antes, pero tiene costo; conviene medir el equilibrio.

Un CSP finito resuelto por retroceso exhaustivo es completo. Encontrar una asignación factible no implica minimizar una preferencia. Para un problema de optimización con restricciones puede usarse ramificación y acotación: conservar la mejor solución y podar asignaciones cuya cota inferior ya no pueda mejorarla. La calidad de una cota determina la poda, del mismo modo que una heurística informa A^* .

Errores frecuentes: modelar como valor especial «sin asignar» y permitirlo en una solución; modificar dominios sin restaurarlos al retroceder; aplicar MRV sobre tamaños originales en vez de dominios actuales; considerar que consistencia de arco garantiza una solución global; y ocultar una restricción dura dentro de una penalización que el optimizador puede aceptar.

7.4.7. Ejemplo práctico guiado: planificación de recorridos de inspección

Una organización debe planificar un turno de 240 minutos para un equipo. Hay cuatro inspecciones: A requiere 35 minutos y prioridad 8; B , 50 y prioridad 10; C , 30 y prioridad 5; D , 45 y prioridad 7. Los tiempos de traslado dependen del orden. B solo está disponible durante los primeros 120 minutos; D debe realizarse después de A porque utiliza su diagnóstico; y toda inspección iniciada debe completarse dentro del turno.

Hay dos decisiones relacionadas pero distintas:

1. **Factibilidad y selección:** qué inspecciones se realizan y en qué condiciones.
2. **Secuenciación:** en qué orden se visitan para reducir traslado o maximizar prioridad atendida.

Una formulación CSP puede usar variables $seleccion_i$, $inicio_i$ y $posicion_i$.

Sus dominios contienen valores booleanos, instantes discretizados y posiciones. Las restricciones imponen no solapamiento, disponibilidad de B , precedencia $A < D$ y duración total. Si todas las tareas deben realizarse y las restricciones son incompatibles, el sistema debe declarar que no existe solución; no puede prolongar implícitamente el turno.

Si algunas inspecciones son opcionales, el objetivo puede maximizar

$$F(x) = \sum_i prioridad_i seleccion_i - \lambda tiempoTraslado(x),$$

siempre que λ tenga interpretación y las restricciones de seguridad permanezcan duras. El estado de una búsqueda local es una lista ordenada de inspecciones seleccionadas. Los vecinos pueden intercambiar dos visitas, invertir un segmento, insertar una omitida o retirar una visita. Cada vecino se reevalúa incluyendo traslado, servicio y ventanas.

Desarrollo guiado: primero se ejecuta propagación de restricciones. La ventana de B reduce sus posiciones posibles; la precedencia elimina órdenes con D antes de A ; el límite de turno descarta combinaciones demasiado largas. Sobre las secuencias factibles se aplica ascenso de colinas con varios reinicios. Se registra valor inicial, valor final, inspecciones cubiertas, tiempo total y razón de parada. Luego se aplica recocido con el mismo número de evaluaciones para observar si aceptar intercambios temporalmente peores permite mejorar el resultado.

Supóngase que el orden inicial $A - C - D - B$ incumple la ventana de B . No debe recibir simplemente una puntuación algo menor y competir como si fuera válido: se repara moviendo B o se rechaza. El orden $B - A - D - C$ respeta precedencia, pero puede superar 240 minutos según traslados. Solo después de confirmar factibilidad se compara su valor con $B - A - C - D$ u otras alternativas.

El informe final incluye la formulación, tabla de tiempos, restricciones, operador de vecindad, presupuesto de evaluación, semillas y distribución de resultados. Debe distinguir «mejor solución encontrada» de «solución óptima». Para demostrar optimalidad sería necesario enumerar todas las soluciones factibles o aplicar un método exacto con cota verificable. En operación real, además, tiempos inciertos, nuevas urgencias y varios equipos convertirían el problema en una decisión dinámica; el modelo estático constituye una aproximación inicial, no un plan inmune a cambios.

Errores comunes y límites del capítulo

Los errores siguientes atraviesan varias estrategias:

- confundir estado con nodo y eliminar una ruta mejor porque «el nodo ya apareció»;
- medir longitud cuando la pregunta exige costo, o sumar magnitudes con unidades incompatibles;
- afirmar completitud u optimalidad sin mencionar ramificación, signo de costos, condición de objetivo y propiedad heurística;
- comparar algoritmos sobre instancias, filtros u órdenes de sucesores diferentes;
- contar generados en un algoritmo y expandidos en otro;
- interpretar la salida como verdad del mundo en vez de óptimo del modelo;

- usar una heurística calculada con información futura o con el costo exacto que se pretende estimar;
- tratar una solución local estocástica como óptimo demostrado;
- convertir restricciones duras en penalizaciones sin analizar violaciones;
- inferir una ruta vial a partir de un grafo agregado de zonas OD.

Los métodos estudiados suponen que el modelo permanece fijo durante una ejecución. Si una calle se cierra, cambia el tiempo de viaje o llega una inspección urgente, puede ser necesario volver a planificar. Si las transiciones son inciertas o las acciones producen observaciones parciales, el espacio de estados debe ampliarse o reemplazarse por modelos de decisión bajo incertidumbre. Si la función objetivo contiene consecuencias sociales, la optimización técnica no decide por sí sola qué pesos son legítimos. Una respuesta responsable documenta alcance, cobertura de datos y personas o fenómenos no representados.

Síntesis

Una búsqueda combina representación y estrategia. El grafo de estados expresa posibilidades; el árbol de búsqueda expresa caminos considerados. Frontera, conjunto explorado y mejor costo controlan el orden y la repetición. BFS privilegia profundidad; DFS, memoria; DLS impone un horizonte; IDS combina profundidad creciente y bajo espacio; UCS optimiza costos no negativos. A* añade una cota del costo restante y conserva optimalidad bajo condiciones precisas; la búsqueda voraz sacrifica esa garantía por dirección más agresiva.

La búsqueda local abandona la reconstrucción de caminos para mejorar configuraciones con poca memoria. Ascenso de colinas, recocido simulado y algoritmos evolutivos ofrecen soluciones aproximadas, por lo que necesitan múltiples ejecuciones y baselines. Los CSP hacen explícitas variables, dominios y restricciones y aprovechan propagación y órdenes de elección. En todos los casos, las garantías pertenecen a un modelo. En MOV-02, ese modelo es un grafo OD de zonas TLC sostenido por viajes observados, no una red de calles.

Glosario

Acción: operación aplicable a un estado que produce una transición.

Admisibilidad: propiedad $h(n) \leq h^*(n)$; la heurística no sobreestima el costo óptimo restante.

Aptitud: medida con la que un algoritmo evolutivo evalúa y selecciona candidatos.

Camino: secuencia de estados conectados por transiciones legales.

Compleitud: garantía de encontrar una solución cuando existe, bajo supuestos declarados.

Consistencia: propiedad heurística $h(n) \leq c(n, n') + h(n')$ para cada transición.

Costo acumulado $g(n)$: suma de costos desde el estado inicial hasta el nodo n .

Dominio: conjunto de valores permitidos para una variable de un CSP.

Estado: descripción suficiente de una configuración relevante del problema.

Estado repetido: estado alcanzado por más de un camino o más de una vez.

Expandir: retirar un nodo de la frontera y generar sus sucesores.

Factor de ramificación: cantidad de sucesores de un nodo; puede usarse un máximo o un valor efectivo.

Frontera: colección de nodos generados pendientes de expansión.

Función heurística $h(n)$: estimación del costo mínimo restante hasta un objetivo.

Grafo OD: grafo dirigido cuyas aristas representan pares origen-destino observados; no implica una red vial.

Meseta: región de estados vecinos con igual valor en búsqueda local.

Nodo de búsqueda: registro que contiene un estado y metadatos como padre, acción, profundidad y costo.

Optimalidad: garantía de devolver una solución de costo mínimo según la función declarada.

Reapertura: reinserción de un estado explorado cuando se descubre un camino de menor costo.

Restricción: relación que determina combinaciones permitidas de valores.

Solución: camino que alcanza un objetivo o asignación completa que satisface restricciones, según el problema.

Vecindario: conjunto de candidatos obtenibles mediante un movimiento local.

Preguntas de revisión y discusión

1. ¿Por qué un espacio con cuatro estados puede generar un árbol de búsqueda infinito? Construya un ejemplo.
2. Explique con sus palabras la diferencia entre generar, insertar y expandir un nodo. ¿Qué métrica informaría en un experimento?
3. ¿Qué información adicional debe contener un estado de ruta si existen ventanas horarias?
4. Demuestre por qué BFS no minimiza tiempo cuando las aristas tienen costos diferentes.
5. ¿En qué condiciones DFS es una elección razonable pese a no ser óptima?
6. ¿Por qué DLS debe distinguir corte de fracaso? Dé un caso donde confundirlos produzca una conclusión falsa.
7. Explique por qué la repetición de niveles en IDS no cambia su orden temporal $O(b^d)$.
8. ¿Por qué UCS comprueba el objetivo al extraer y no al generar?
9. Construya un grafo donde la búsqueda voraz encuentre una solución mucho más cara que UCS.
10. Demuestre que una heurística consistente con $h(g) = 0$ es admisible a lo largo de un camino óptimo.
11. ¿Qué debe hacer A* con una heurística admisible pero inconsistente?
12. ¿Por qué una alta correlación entre heurística y costo verdadero no demuestra admisibilidad?
13. Compare $\max(h_1, h_2)$ y $h_1 + h_2$ cuando ambas heurísticas son admisibles. ¿Qué condición adicional necesita la suma?
14. ¿Cómo afectan máximos locales, mesetas y crestas al ascenso de colinas?

15. ¿Qué sucede con recocido simulado si se cambia el costo de minutos a segundos y se conserva el mismo calendario de temperatura?
16. Distinga una restricción dura de una preferencia blanda en la planificación de inspecciones.
17. ¿Por qué consistencia de arco no garantiza por sí sola una solución completa de un CSP general?
18. Explique por qué una secuencia de zonas obtenida en MOV-02 no es una ruta de conducción.
19. Si el costo OD es tiempo mediano, ¿por qué la distancia entre centroides no puede sumarse directamente a g ?
20. ¿Qué análisis de sensibilidad haría antes de confiar en un camino obtenido tras filtrar aristas por soporte?

Actividad integradora del capítulo

Propósito. Comparar familias de búsqueda sobre una formulación común y defender una decisión metodológica con garantías, evidencia y límites.

Se propone un problema de planificación de cinco a ocho inspecciones con ubicaciones agregadas, duraciones, prioridades, precedencias y una ventana temporal. Cada equipo define un conjunto pequeño y completamente auditable. No se requieren datos de movilidad ni código vial; los tiempos de traslado pueden darse en una matriz didáctica.

Fase 1, formulación. Definir estados, acciones, transición, objetivo y costo para una búsqueda de caminos. Formular también variables, dominios y restricciones como CSP. Explicar qué detalles se omiten y presentar al menos un par de estados que parezcan iguales pero deban distinguirse por tiempo o tareas pendientes.

Fase 2, referencia exacta. Sobre una versión reducida, aplicar costo uniforme y obtener una solución de referencia. Informar orden de expansión, mejoras de costo, máximo de frontera y camino. Si se usa un límite o se poda, justificar que no elimina la solución óptima.

Fase 3, búsqueda informada. Diseñar una heurística mediante relajación, por ejemplo ignorar precedencias y usar la suma de mínimos costos pendientes. Probar objetivo cero, unidad, admisibilidad o, si no puede demostrarse, declarar la ausencia de garantía. Comparar A* con UCS sobre las mismas instancias.

Fase 4, optimización local y restricciones. Representar una solución completa como secuencia, definir al menos dos movimientos de vecindario y comparar ascenso de colinas con reinicios frente a recocido simulado bajo igual número de evaluaciones. Usar propagación del CSP para rechazar o reparar candidatos inviables. Ejecutar varias semillas y reportar mediana, mejor, peor y proporción de soluciones factibles.

Fase 5, argumentación. Recomendar una estrategia para tres escenarios: memoria muy limitada, exigencia de optimalidad y necesidad de una respuesta rápida aproximada. La recomendación debe citar supuestos y no basarse solo en el menor tiempo observado.

Producto esperado: informe breve con formulación formal, pseudocódigo adaptado en términos del dominio, tabla comparativa, trazas de una instancia, análisis de errores y una sección de límites. No se acepta una tabla de tiempos sin reconstrucción de caminos y validación de costos.

Rúbrica orientativa:

Dimensión	Evidencia esperada
Representación	Estado suficiente, acciones legales, objetivo y unidades inequívocas.
Corrección	Caminos válidos, costos recalculados y restricciones verificadas.
Garantías	Completitud y optimalidad asociadas a supuestos concretos.
Comparación	Mismas instancias, presupuesto y definiciones de métricas.
Heurística	Origen conceptual, propiedades comprobadas y costo de evaluación.
Búsqueda local	Varias semillas, factibilidad y distribución de calidad.
Comunicación	Conclusiones limitadas al modelo y errores discutidos.

La actividad cierra la transición entre representación y decisión. Un algoritmo no compensa una formulación incorrecta: únicamente explora con mayor o menor eficacia las alternativas que el modelo hizo visibles.